Data Structure

Hoda Osama

Variables

Variables are containers for storing data values.

In C, there are different types of variables (defined with different keywords), for example:

- int - stores integers (whole numbers), without decimals, such as 123 or -123
- float - stores floating point numbers, with decimals, such as 19.99 or -19.99
- char - stores single characters, such as 'a' or 'B'. Char values are surrounded by single quotes

Arrays

Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.

Int Arr[5];

	Arr[0]	Arr[1]	Arr[2]	Arr[3]	Arr[4]
Array elements	100	200	300	400	500
Array indices	0	1	2	3	4

Structures

Structures (also called structs) are a way to group several related variables into one place. Each variable in the structure is known as a member of the structure.

Syntax of struct

```
struct structureName {
  dataType member1;
  dataType member2;
  ...
};
```

Structures

Create a struct variable with the name "s1":

```
struct myStructure {
  int myNum;
  char myLetter;
};

int main() {
  struct myStructure s1;
  return 0;
}
```

Structures

Access Structure Members

```
struct myStructure {
  int myNum;
  char myLetter;
};

int main() {
  struct myStructure s1;  // Assign values to members of s1
  s1.myNum = 13;
  s1.myLetter = 'B';

  printf("My number: %d\n", s1.myNum);
  printf("My letter: %c\n", s1.myLetter);

  return 0;}
```

Structures

Nested structure

```
struct Employee
{
  int employee_id;
  char name[20];
  int salary;
};
  struct Organisation
{
   char organisation_name[20];
   char org_number[20];
   struct Employee emp;
};
```

```
int main()
{
  struct Organisation org;
  org.emp.employee_id = 101;
  strcpy(org.emp.name, "Robert");
  org.emp.salary = 400000;
  strcpy(org.organisation_name, "GeeksforGeeks");
  strcpy(org.org_number, "GFG123768");

  // Printing the details
  printf("Organisation Name : %s\n", org.organisation_name);
  printf("Organisation Number : %s\n",org.org_number);
  printf("Employee id : %d\n", org.emp.employee_id);
  printf("Employee name : %s\n", org.emp.name);
  printf("Employee Salary : %d\n", org.emp.salary);
}
```

Pointers

Pointers store address of variables or a memory location.

```
// General syntax
datatype *var_name;

int *ptr;
```

Pointers

To access address of a variable to a pointer, we use the unary operator & (ampersand) that returns the address of that variable

```
#include <stdio.h>

int main()
{
    int x;

    // Prints address of x
    printf("%p", &x);

    return 0;
}
```

Pointers

One more operator is unary * (Asterisk) which is used for two things :

1. To declare a pointer variable: When a pointer variable is declared in C/C++, there must be a * before its name.

```
#include <stdio.h>
int main()
{
    int x = 10;
    int *ptr;

    // & operator before x is used to get address
    // of x. The address of x is assigned to ptr.
    ptr = &x;

    return 0;
}
```

Pointers

2. To access the value stored in the address we use the unary operator (*) that returns the value of the variable located at the address specified by its operand. This is also called Dereferencing.

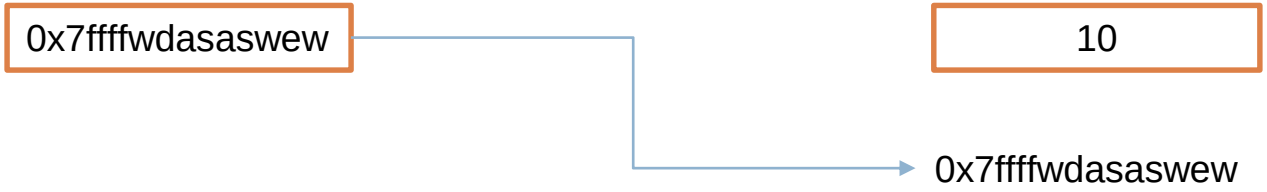
```
#include <stdio.h>
int main()
{
    int Var = 10;
   // A pointer variable that holds address of var.
   int *ptr = &Var;
   // This line prints value at address stored in ptr.
   // Value stored is value of variable "var"
   printf("Value of Var = %d\n", *ptr);
   // The output of this line may be different in different
   // runs even on same machine.
   printf("Address of Var = %p\n", ptr);

   *ptr = 20; // Value at address is now 20
    printf("After doing *ptr = 20, *ptr is %d\n", *ptr);

   return 0;
}
```

Pointers

```
Int var =10;
Int *p;
p=&var
```




```
printf("%d", var);          // equal to printf("%d", *p);

*p = *p+ 100;       // 10+100 = 110 (X equals 110)
```

Pass by value

The pass by value method copies the variable

```
#include <stdio.h>
void test(int x) {
        x++;
return;
}
int main () {
          int x=6;
          test(x);
          printf(“%d”,x);
return 0;
}
```

Pass by refrence

The pass by reference method copies the address of an argument

```
#include <stdio.h>
void test(int *x) {
        (*x)++;
return;
}
int main () {
          int x=6;
          test(&x);
          printf(“%d”,x);
return 0;
}
```

Structure Pointer

The structure pointer points to the address of a memory block where the Structure is being stored.

Initialization of the Structure Pointer

```
struct structure_name *ptr = &structure_variable;
```

Access Structure member using pointer:

There are two ways to access the member of the structure using Structure pointer:

- Using (*) asterisk or indirection operator and dot (.) operator.
- Using arrow (->) operator or membership operator.

Structure Pointer

```
#include <stdio.h>
  struct Subject
{
    char sub_name[30];
    int sub_id;
    char sub_duration[50];
    char sub_type[50];
};
```

```
int main()
{
    struct Subject sub; // declare the Subject variable
    struct Subject *ptr; // create a pointer variable (*ptr)
    strcpy (sub.sub_name, " Computer Science");
    sub.sub_id = 1201;
    strcpy (sub.sub_duration, "6 Months");
    strcpy (sub.sub_type, " Multiple Choice Question");

    // print the details of the Subject;
    printf (" Subject Name: %s\t ", (*ptr).sub_name);
            printf (" \n Subject Id: %d\t ", (*ptr).sub_id);
        printf (" \n Duration of the Subject: %s\t ", (*ptr).sub_duration);
           printf (" \n Type of the Subject: %s\t ", (*ptr).sub_type);

    return 0;
}
```

Structure Pointer

```
#include <stdio.h>
  struct Subject
{
    char sub_name[30];
    int sub_id;
    char sub_duration[50];
    char sub_type[50];
};
```

```
int main()
{
    struct Subject sub; // declare the Subject variable
    struct Subject *ptr; // create a pointer variable (*ptr)
    strcpy (sub.sub_name, " Computer Science");
    sub.sub_id = 1201;
    strcpy (sub.sub_duration, "6 Months");
    strcpy (sub.sub_type, " Multiple Choice Question");

    // print the details of the Subject;
    printf (" Subject Name: %s\t ", ptr->sub_name);
            printf (" \n Subject Id: %d\t ", ptr-> sub_id);
        printf (" \n Duration of the Subject: %s\t ", ptr-> sub_duration);
           printf (" \n Type of the Subject: %s\t ", ptr-> sub_type);

    return 0;
}
```

Memory allocation

- Static memory allocation:

Store variables in stack and it happens at compile time based on variable definitions.

- Dynamic memory allocation:

Allocation of memory storage (all available unused memory called the heap) for use in a computer program during the runtime.

Static memory allocation

- int arr[10];

0x804837c	
0x8049588	
0x8049688	
0x8049690	
0x8049698	

Dynamic memory allocation

- **In C:**

```
int  *X = malloc(sizeof (int));  // X is pointer that points to single int.
free(X);            // location in heap that is pointed by pointer X is de-allocated
int  *X_arr = malloc (sizeof(int)*10);          //pointer to array
free(X_arr);                                    // array is de-allocated
```

- **In C++:**

```
int  *X = new int;
delete X;
int  *X_arr = new int[10];
delete[ ] X_arr;
```